

ATP Cross-Verification of the Mizar MPTP Challenge Problems

Josef Urban^{1,*} and Geoff Sutcliffe²

¹ Dep't of Theoretical Computer Science, Charles University in Prague

² Dep't of Computer Science, University of Miami

Abstract. Mizar is a proof assistant used for formalization and mechanical verification of mathematics. The main use of Mizar is in the development of the Mizar Mathematical Library (MML), in which proofs are verified by the Mizar proof checker. The Mizar proof checker has a quite complex implementation, and also lacks the ability to print out detailed atomic proof steps in a format that is easy to verify by an independent proof-checking tool. This can raise concerns about the correctness of the MML. This paper describes how a Mizar-to-ATP translation (the MPTP system), ATP verification tools (the GDV system), and Automated Theorem Proving (ATP) systems, have been used for an independent cross-verification of a part of the MML.

1 Introduction, Motivation, and Related Work

Mizar [Rud92, RT99] is a proof checker based on first-order classical logic and set theory. The main use of Mizar is in the development of the Mizar Mathematical Library (MML)¹, a large library of formally verified mathematics, allowing the formalization of more and more advanced mathematical theories.

1.1 Motivation: Verification of Mizar Proofs

Starting in the 1990s with John Harrison's Mizar Mode for HOL [Har96], various versions of the Mizar declarative proof formalism have been developed [Zam99, Sym99, Wen99]. However, so far, there has been no independent verification of Mizar proofs. This (among other effects) prevents trusted semantic communication between Mizar and other systems. Such inter-system communication is useful for very large formalization efforts, e.g., the verification of the proof of Kepler Conjecture [Hal04]. In this context, communication between LCF-based systems has recently become popular [OS06, McL06].

While Mizar's reasoning formalism is quite simple and intuitive, it is mainly Mizar's implementation of its "atomic inference step"² that makes Mizar emulation complicated. Although the basic idea motivated by the notion of obviousness [Rud87, Dav81] is relatively easy to explain [Wie00], and has even been

* Supported by a Marie Curie International Fellowship within the 6th European Community Framework Programme.

¹ <http://mizar.uwb.edu.pl/library/>

² Denoted with the keyword **by** in Mizar proofs - see Section 2.2.

re-implemented in Harrison’s Mizar Mode for HOL, there have been a number of additions in Mizar (e.g., interaction with Mizar’s type system, built-in computer algebra [NB02, NB04], etc.) that make even a Mizar implementation of a detailed documentation mode for this procedure quite expensive. On the other hand, Mizar’s classical first-order foundations are very close to the formalism used in today’s efficient Automated Theorem Proving (ATP) systems. Translation of the MML to ATP formats (like TPTP [SS98]) is the goal of the MPTP (Mizar Problems for Theorem Proving) project. First experiments [Urb06a, Urb04] indicated that current ATP systems are able to verify an overwhelming majority of Mizar’s atomic inference steps – above 99% of 18,429 problems generated from 48 initial Mizar articles [Urb06a] – without any low-level guidance. Given this capability for verification of atomic inference steps, the next logical step (described in this paper) is to develop techniques extending this to ATP-based verification of whole (sometimes quite long) Mizar proofs, with the potential target of ATP-based verification of the whole MML.

This paper describes how the Mizar-to-ATP translation has been enhanced (Section 2.2) so that the translated Mizar atomic inference steps of a Mizar proof are linked together to form a TPTP format derivation that reflects the the higher-level (natural deduction) Mizar reasoning structure. The derivation can then be verified using tools for verification of ATP systems’ derivations, to provide verification of the Mizar proof. As necessary byproducts, the TPTP language has been extended for encoding derivations containing *assumptions* (Section 2.1), correctness conditions for such derivations have been designed and implemented in the GDV verification tool (Section 3), and an explanation of the Mizar natural deduction constructs has been provided in terms of this simpler formalism.

1.2 Related Verification Work

A list of previous mutual proof assistant verifications would probably be quite long (the translations between HOL Light and Isabelle are cited above). However, they are usually done on systems with low-level “proof object” capabilities, or systems in which addition of such capability is relatively easy, thanks to their “minimal logical kernel” design. The Otter ATP system has the “detailed proof object” option, and the generated Otter proof objects can be verified by the IVY system [MS00]. A recent notable use of ATP systems for certification of safety properties of NASA software is described in [DFS06].

1.3 Target: Verification of the MPTP Challenges

The methods developed for the ATP verification of Mizar theorems have one immediate application and nontrivial target: The set of ATP problems serving as the MPTP Challenges.³ This is a set of 252 related mathematical problems translated from the Mizar library by the MPTP system, to create a benchmark for measuring the capability of ATP systems to work in large theories. Some

³ <http://www.cs.miami.edu/~tptp/MPTPChallenge/>

of these problems have quite long Mizar proofs, making their direct solution by ATP systems quite difficult. As of May 2007, only 199 of the 252 problems (79%) have been solved by any ATP system,⁴ leaving open questions about the provability of the remaining 53 problems. Since errors have been encountered before, both in the Mizar verifier and in the MPTP translation, an independent verification of the problems would be useful. This has been carried out and is discussed in Section 4.

2 Translation of Mizar Proofs to TPTP Derivations

2.1 Extending the TPTP Format for Assumptions

Assumptions (*suppositions* in the original Jaskowski's [Jas34] terminology) are unproved propositions introduced (assumed to be true) locally at some part of a proof, coupled with an elimination (discharge) rule for making the results of such subproofs unconditionally valid. A common implicit use of these introduction/discharge rules in refutational ATP systems is the introduction of the negated conjecture, the derivation of a contradiction, and the discharge of the negated conjecture at that point, i.e., the argument that “ $\sim$ conjecture $\vdash$ $\$false$ ” is equivalent to “ $\vdash \sim$ conjecture $\Rightarrow$ $\$false$ ”, and hence to “ $\vdash$ $\sim$ conjecture”. Other examples from the ATP world include explicit (SPASS-like) splitting [WBH⁺02], and the formalisms used in tableaux systems.

In order to record information about the use of assumptions in derivations, a new formulae role **assumption** has been added to the TPTP language, and a semantic grammar rule has been added for the source information recorded with every inferred formula. The semantic grammar rule specifies how an arbitrary list (treated as set by the GDV checks, see Section 3) of assumptions can be recorded with a formula, to keep track of the assumptions upon which the formula depends. The existing semantic grammar rules were already sufficient for recording the discharge of an arbitrary list of assumptions. These semantic grammar rules make it possible to record and verify that a derivation is independent of any assumptions introduced in it. These changes are part of the TPTP language specification in TPTP v3.3.0. An example of an assumption introduced into a derivation, an inferred formula that depends on recorded assumptions, and an inference involving the discharge of assumptions (modified from an MPTP challenge problem), are:

```
fof(e1,assumption, subset(c1,c2), introduced(assumption)).

fof(i3,plain, c2 = set_union2(c1,set_diff(c2,c1)),
  inference(conclusion,[status(thm),assumptions([dt_c1,e1]],[e3,i4])).

fof(i2,plain, (subset(c1,c2) => c2 = set_union2(c1,set_diff(c2,c1))),
  inference(discharge_asm,[status(thm),assumptions([dt_c1]),
    discharge_asm(discharge,[e1]],[e1,i3])),
```

⁴ See <http://www.cs.miami.edu/~tptp/MPTPChallenge/Results/SVGResults.html>

This “recording” approach is more effective than adding assumptions as antecedents of implications to their descendants. Adding assumptions as antecedents would significantly increase the difficulty of the ATP problems built from the Mizar atomic inference steps, possibly preventing completion of the cross-verification. These syntactic additions are sufficient for translation of Mizar proofs into efficiently verifiable TPTP derivations (see Section 2.2). The consequence of this decision is a need to ensure that recorded assumptions are correctly propagated and discharged. This is done by GDV, as described in Section 3.

2.2 Mizar Natural Deduction

Quite often it is claimed that Mizar uses Jaskowski-style natural deduction. Even though Jaskowski’s formalism is certainly related to the Mizar formalism, there are too many differences to take Jaskowski’s original paper [Jas34] (or its more accessible version [Pel99]) as an accurate account of the Mizar formalism that could be used for explaining its translation to classical first-order logic form. Quite a precise overview of the Mizar formalism is given in [Har96], for the purpose of its implementation in HOL. In what follows the use of Gentzen-like introduction/elimination terminology, and comparison with various Gentzen-style calculi, is avoided. This is mainly to keep the exposition simple, and to avoid possible confusion caused by different flavors and semantics of those calculi used in different systems (the interested reader is referred to the detailed overview of these issues in [Pel99]).

The Mizar parser translates Mizar articles into a semantic XML format [Urb06b], which is used internally by the Mizar proof checker, and also by various external tools such as the Mizar-to-TPTP (MPTP) translator. The XML format is specified in the RELAX NG schema language.⁵ The XML format specification is always up-to-date, because it is generated in a semi-automated way from the Mizar source code, and because XML validation can be used to check the XML representation of articles. For these reasons, the XML format specification has been used in this work as a complete up-to-date abstract syntax for the Mizar formalism.⁶ The part of the XML format specification describing Mizar natural deduction proofs is (with minor simplification) as follows:

```
## Theorem as a proposition with justification.
JustifiedTheorem = element JustifiedTheorem {Proposition, Justification}
Justification = ( Inference | Proof | SkippedProof )
## By and From encode the atomic inference steps done by Mizar.
Inference = ( By | From | element ErrorInf { empty } )
## By encodes one simple justification from zero or more references.
By = element By { Ref* }
## From encodes a simple justification involving Mizar schemes
From = element From { Ref* }
## Proofs (of BlockTheorem) encode ND reasonings consisting of many steps.
Proof = element Proof {BlockTheorem, Reasoning }
## Reasoning is a series of skeleton and auxiliary items (steps)
```

⁵ <http://relaxng.org/>

⁶ Available at <ftp://mizar.uwb.edu.pl/pub/version/doc/xml/Mizar.html>

```

## optionally finished by reasoning per cases.
Reasoning = ( ( SkeletonItem | AuxiliaryItem )*, PerCasesReasoning? )
## Skeleton items are the steps that change the current thesis, the
## new Thesis is printed explicitly after them.
SkeletonItem = ((Let|Conclusion|Assume|Given|Take|TakeAsVar), Thesis)
## Auxiliary items are items which do not change thesis.
AuxiliaryItem = (JustifiedProposition|Consider|Set|Reconsider|
                  DefFunc|DefPred)

```

The **By** and **From** justifications denote atomic inference steps that are simple enough for the Mizar proof checker. There is no substructure to atomic inference steps, and they are translated directly as first-order inferences, e.g.,

```

A1: p;                fof(a1,plain,p).
A2: p implies q;      fof(a2,plain,p => q).
A3: q by A1, A2;      fof(a3,plain,q,inference(by,[status(thm)],[a1,a2])).

```

The Mizar inferences on the left translate into the TPTP format derivation on the right. (The translation can also add parent formulae that encode “background” information used implicitly by the Mizar proof checker, e.g., the Mizar type hierarchy.) The interesting case is when a full **Proof** is used in Mizar. This is done when the Mizar proof checker (used for the **By** and **From** constructs) is not strong enough, and the goal (**thesis** in Mizar terminology) either needs to be simplified by various natural deduction steps (**Let**, **Conclusion**, **Assume**, **Given**, **Take**, **TakeAsVar**, **PerCasesReasoning**), or the proof context needs to be augmented by a new lemma (**JustifiedProposition**), a new constant satisfying some property (**Consider**, **Reconsider**), or a useful macro (**Set**, **DefFunc**, **DefPred**). The following subsections explain these parts of Mizar reasoning in more detail.

Explanation of the Mizar Assumption and Conclusion steps

```

;; Example 1 (Mizar code)
Lemma1: p implies q;          ;; this has been proved already
Lemma2: q implies r;          ;; this has been proved already
Goal1: p implies q & r
proof                          ;; (Proof) thesis (Goal1) is: p implies q & r
  assume E1: p;                ;; (Assumption) thesis (T1) is: q & r
  thus E2: q by Lemma1,E1;      ;; (Conclusion) thesis (T2) is: r
  thus E3: r by Lemma2,E2;      ;; (Conclusion) thesis (T3) is: verum
end;

```

An **Assume** (keyword **assume**) step is used when the thesis can be considered to be an implication. Very much like in the original Jaskowski’s formalism, it adds the antecedent (**p** in Example 1) of the **thesis** (**p implies q & r**) to the list of valid propositions available for further use in this block, and changes the **thesis** of the block to its consequent (**q & r**). As can be seen from Example 2, the Mizar notion of *normalization* (see, e.g., [Wie00] for details) is quite strong. It identifies the proposition “not **p**” with the proposition “**p implies contradiction**”. In this feature Mizar certainly differs from Jaskowski’s original treatment. The

advantage is that it turns a proof by contradiction into just a special case of proof by supposition.

```
;; Example 2 (Mizar code)
Lemma3: p implies q;           ;; this has been proved already
Lemma4: not q;                 ;; this has been proved already
Goal2: not p;
proof                          ;; (Proof) thesis (Goal2) is: not p
  assume E1: p;                ;; (Assumption) thesis (T1) is: contradiction
  E2: q by E1, Lemma3;         ;; (JustifiedProposition) thesis unchanged
  thus E3: contradiction      ;; (Conclusion) thesis (T2) is: verum
    by E2, Lemma4;
end;
```

The **Conclusion** (keyword **thus**) step is used to prove (a part of) the current thesis. Mizar checks that the proposition that is proved is either equal (in a normalized form) to the current thesis (this is the E3 case in the first example), or a conjunct of the current thesis (this is the case E2 in the first example). This way, the thesis is reduced by each **Conclusion** step, and upon encountering the **end** keyword, Mizar checks that the thesis is just **verum** (*true*). It should be noted that here again the Mizar normalization is responsible for seemingly unrelated deconstructions of the thesis. For example, if the thesis is “**p iff q**”, a **Conclusion** step proving “**p implies q**” is legal, transforming the thesis to “**q implies p**”.

Translation of the Mizar Assumption and Conclusion steps

```
% Translation of Example 1 into extended TPTP derivation
fof(lemma1,plain, p => q).
fof(lemma2,plain, q => r).
fof(e1,assumption,p,introduced(assumption)).
fof(e2,plain,q,
  inference(mizar_by,[status(thm),assumptions([e1]),[lemma1,e1])).
fof(e3,plain,r,
  inference(mizar_by,[status(thm),assumptions([e1]),[lemma2,e2])).
fof(t3,plain,$true,introduced(tautology)).
fof(t2,plain,r,
  inference(conclusion,[status(thm),assumptions([e1]),[t3,e3])).
fof(t1,plain,q & r,
  inference(conclusion,[status(thm),assumptions([e1]),[t2,e2])).
fof(goal1,plain,p => q & r,
  inference(discharge_asm,[status(thm),discharge_asm(discharge,[e1]),
    [e1,t1])).
```

The translation of Example 1 to a TPTP derivation is quite straightforward. Note that the **theses** are collected in a backwards manner and used to justify their **thesis**-predecessors, either by a **Conclusion** inference (usually involving some other lemmas), or by a **discharge_asm** inference involving a assumption.

This ends at the top-most thesis, i.e., the goal of the proof. The bottom-most **\$true** thesis could be trivially removed, but retaining it makes the translated proofs more faithfully correspond to their Mizar counterparts. Note how the assumptions are propagated and finally discharged.

Explanation of the Mizar Let, Consider, and Take Steps

```
;; Example 3 (Mizar code)
Lemma1: for x being set holds p(x)          ;; this has been proved already
Lemma2: for x being set holds p(x) => q(x) ;; this has been proved already
Goal1: for x being set holds q(x)
proof
  let new_constant be set;                ;; (Let) thesis (T1) is: q(new_constant)
  E1: p(new_constant) by Lemma1;          ;; (JustifiedProposition) thesis unchanged
  thus E2: q(new_constant) by Lemma2,E1;  ;; (Conclusion) thesis (T2) is: verum ($true)
end;

;; Example 4 (Mizar code)
Lemma3: ex x being set st p(f(x))          ;; this has been proved already
Lemma4: for x being set holds p(x) => q(x) ;; this has been proved already
Goal2: ex x being set st q(x);
proof
  consider new_constant being set such    ;; (Consider) thesis unchanged, adds new_constant
  that E1: p(f(new_constant)) by Lemma3;  ;; of type set and proposition p(f(new_constant))
  take f(new_constant);                   ;; (Take) thesis (T1) is: q(f(new_constant))
  thus E2: q(f(new_constant)) by E1, Lemma4; ;; (Conclusion) thesis (T2) is: verum ($true)
end;
```

A **Let** (keyword **let**) step is used when the normalized form of thesis is a universally quantified formula. This step introduces a new constant of the proper type (**set** is used above), which is available from then on in the proof block. (In Mizar the new constants are numbered serially, so instead of the **new_constant** used in Example 3, it would be just, e.g., **c1**) The thesis is instantiated with this new constant (to **q(new_constant)** above). This corresponds to the common mathematical expression “let’s have an arbitrary but fixed *x*”, and provided that the constant is fresh (which is guaranteed in Mizar by the serial numbering), the original universally quantified thesis follows from the instantiated thesis. (This is the standard theorem about constants in Hilbert’s predicate calculus.)

The **Consider** (keyword **consider**) step has no effect on the proof thesis, although it is in some sense dual to the **Let** construct. Given a normalized existentially quantified statement (like **Lemma3** in Example 4), Mizar creates a fresh constant of the appropriate type and instantiates with it the existential statement. The new constant and the instance are available for further use in the proof block. Provided that the constant is fresh, this is again just a conservative extension of the original theory.

The **Take** (keyword **take**) step is used to select a suitable term for instantiating an existentially quantified thesis (**f(new_constant)** is taken in Example 4). The type of the term has to correspond to the type of the existential variable. The original thesis then follows from the instantiated one. A minor variant of this syntax is the **TakeAsVar** step (in Example 4 it would be “**take another_new_constant = f(new_constant);**”), which additionally creates a fresh constant of the same type as the selected term, adds their equality to the proof context, and instantiates the thesis with this fresh constant.

Translation of the Mizar Let, Consider, and Take Steps

```

%% Translation of Example~3 into extended TPTP derivation
fof(lemma1,plain, ![X]: (set(X) => p(X))).
fof(lemma2,plain, ![X]: (set(X) => (p(X) => q(X)))).
fof(henkin_ax1, plain, (set(new_constant) => q(new_constant)) =>
    ![X]: (set(X) => q(X)),
    introduced(definition, [new_symbol(new_constant)])).
fof(type_ass1, assumption, set(new_constant), introduced(assumption)).
fof(e1,plain, p(new_constant), inference(mizar_by,[status(thm),
    assumptions([type_ass1])],[type_ass1,lemma1])).
fof(e2,plain, q(new_constant), inference(mizar_by,[status(thm),
    assumptions([type_ass1])],[type_ass1,e1,lemma2])).
fof(t2,plain, $true,introduced(tautology)).
fof(t1,plain, q(new_constant),
    inference(conclusion,[status(thm),assumptions([type_ass1])],[t2,e2])).
fof(t1_1,plain, set(new_constant) => q(new_constant),
    inference(discharge_asm, [status(thm),
        discharge_asm(discharge, [type_ass1])], [type_ass1, t1])).
fof(goal1,plain, ![X]: (set(X) => q(X)),
    inference(let, [status(thm)], [t1_1, henkin_ax1])).

```

This translation of Example 3 demonstrates that it is necessary to deal with Mizar’s typed language, which is done through relativization to the untyped formalism used by ATP systems. The method of translating the **Let** inference step is the introduction of a Henkin (sometimes also called Skolem) axiom (**henkin_ax1** above) partially defining the new constant. These additional axioms are generated by the MPTP translation system from the known (instantiated) thesis (say “**T(constant)**”) as the corresponding implications (“**T(constant) => ![X]:T(X)**”). Axioms of this form (as well as of the dual form used for translation of the **Consider** constructs, i.e., “**?[X]:T(X) => T(constant)**”) extend the existing theory conservatively, provided that the constant is really new. To check this (a bit metalogical) fact (i.e., that Mizar always generates a fresh constant name here), the “**introduced(definition, [new_symbol(...)])**” TPTP status is used for these axioms. This tells GDV to carry out this check independently (see the description of leaf checks in Section 3.2). GDV does not yet implement a structural check verifying that these are really Henkin axioms, i.e., that they have the required form of an instance implying the universal formula (or of the dual existential form). Their correctness is “by construction” in the MPTP translation system, which is quite obvious from the following core of the Prolog code used for this:

```

%% given a Const-ant, and the Instance, generate a Henkin
%% axiom introducing Const
create_henkin_axiom_let(+Const,+Instance,-Henkin_Ax):-
    apply_const_subst((Const/NewVar), Instance, GenInst),
    Henkin_Ax = (Instance => ( ! [NewVar] : GenInst )).

%% replace atom Const everywhere in Fla with Var, yielding Fla1
apply_const_subst((Const/Var),+Fla,-Fla1):-
... [code skipped] ...

```


Given that the relativization method is used for type translation, the **Let** steps additionally have to be complemented by the assumption and corresponding discharge of the new constant's type (see the translation example above).

The translation of Example 4 is not shown here because of space restrictions. The translation of the **Consider** construct is very similar to **Let** – again a corresponding Henkin axiom is added by the MPTP translation system, but the constant's type is derived instead of assumed. The translation of the **Take** (or **TakeAsVar**) steps is straightforward, as they only justify an existential thesis from its instance (which is a standard first-order inference).

Translation of the other Mizar reasoning constructs

The remaining Mizar reasoning constructs not explained so far are: **Given**, **PerCasesReasoning**, **JustifiedProposition**, **Reconsider**, **Set**, **DefFunc**, and **DefPred**. The **Set**, **DefFunc**, and **DefPred** constructs define macros (shorthands for terms and formulae) that are automatically expanded during the translation of formulae, and therefore the macros themselves do not have to be translated.⁷ A **JustifiedProposition** is an arbitrary lemma introduced in the reasoning, and it (similarly to **JustifiedTheorem**) can be justified either by atomic inference steps, or by its own subproof. The **Given** step is a compressed syntax for an **Assume** step followed by a **Consider** step. **PerCasesReasoning** allows several logically complementary alternatives to be explored in Mizar, and it is translatable using the TPTP assumption format. **Reconsider** is used to justify that a term has a certain type, and it introduces a new constant (with the new type) as a shortcut for the term. This is translated as a corresponding equality introducing the new constant (its freshness is again checked by GDV), and a (provable) proposition stating the typing of the new constant. The translation of all these constructs does not require any more extensions of the TPTP formalism, or any new kinds of axioms, and given the explanations in the previous sections, their translation becomes quite routine. Due to the space restrictions, we therefore do not provide detailed examples of translations of these constructs here. Interested readers can consult the MizarTPTP web page (explained in more detail in [UTSP07]), where all (ca. 40000) Mizar theorems are linked to their corresponding TPTP derivations.⁸

3 The GDV Derivation Verifier

GDV [Sut06] is a tool for verifying derivations in the TPTP format. GDV uses structural and semantic techniques. Structural verification checks that inferences have been done correctly in the context of the derivation, e.g., checking that the derivation is acyclic. Semantic verification checks the required semantic properties of inference steps, e.g., checking that an inferred formula is a logical consequence of its parents. As [Sut06] already provides comprehensive coverage of the design and implementation of many aspects of GDV, this section provides

⁷ ATP systems' clausifiers often use similar shortcuts to improve the resulting CNF.

⁸ <http://www.cs.miami.edu/~tptp/MizarTPTP/>

an overview of the GDV process, and details of new aspects of GDV that were used in this work.

3.1 Structural Verification

The structural verifications done by GDV are, for the most part, quite simple. The most basic checks are made first: checking that all the nodes in the derivation DAG are distinctly named, checking that all inference steps are adequately documented and the named parents exist, checking that the derivation is acyclic, i.e., the derivation is a DAG, and checking that refutations have (only) *false* roots. More complex structural checks are made after that: checking that proofs by contradiction are well formed, checking that split refutations are not mutually dependent, and checking that inference chains with assumptions have been recorded correctly.

Other than the checking of assumptions, all the structural checks listed are described in [Sut06]. The checking of inference chains with assumptions was added to GDV for this work - it was not needed previously because GDV had been used only to check derivations from refutation based ATP systems, which do not use assumptions⁹, while the natural deduction style of MML proofs makes extensive use of assumptions. The structural checking of inference chains with assumptions has two aspects. First it is necessary to check that assumptions are propagated from parents to inferred formulae, modulo discharged assumptions. This is done by ensuring that the set of assumptions of the parents, i.e., the parents that are assumptions plus the recorded assumptions of the parents, are a subset of the union of the discharged and recorded assumptions of the inferred formula. Second it is necessary to check that all assumptions are discharged. This is done by checking that no root of the DAG has any remaining recorded assumptions.

Note that these structural checks on assumptions make no examination of the logical formulae involved, so that they do not ensure that the discharges do correctly remove dependence on the assumptions. Semantic verification is used to check that, as is described in Section 3.2.

3.2 Semantic Verification

The technique used in semantic verification is to encode the required semantic relationship between each inferred formula and its parent formulae into logical obligations, in the form of ATP problems. The obligations are discharged by having trusted ATP systems solve the ATP problems. The required semantic relationship between an inferred formula and its parent formulae depends on the intent of the inference rule used. Most commonly an inferred formula is intended to be a theorem (logical consequence) of its parent formulae, but in other cases, e.g., Skolemization and splitting, the inferred formula has a weaker relationship with its parents. This intent is recorded as an SZS [SZS04] status annotation

⁹ However, it is planned to convert the checking of the negated conjectures in refutations to the checking of assumptions form.

in the inference record of each inferred formula in a TPTP format derivation, e.g., in the second formulae in Section 2.1 the SZS status is `thm`, which indicates that the inferred formula is required to be a theorem of its parents. GDV's semantic verification has three phases: leaf verification, rule specific verification, and general semantic verification.

There are two types of leaves in a derivation - leaves that come from the problem solved by the derivation, and leaves that were introduced by the ATP system. Verification of leaves that come from the problem is described fully in [Sut06], based on the idea that leaf axioms must be derivable from problem axioms, and a leaf conjecture must be able to derive the problem conjecture. Leaves are introduced into derivations by ATP systems for a variety of (legitimate) reasons. For this work the reasons that occur are the introduction of tautologies, the introduction of definitions, and the introduction of assumptions. Tautologies are verified by proving (using a trusted ATP system, as explained above) them. Definitions and assumptions do not produce a verification obligation, but definitions are checked to ensure that the defined symbol does not occur in any other non-introduced leaf.

Rule specific verification deals with explicit splitting, as implemented in the ATP system SPASS [WBH⁺02], pseudo-splitting, as implemented in Vampire [RV02] and E [Sch02], application of definitions as implemented in E, and discharging assumptions as found in the MPTP derivations that are the focus of this work. The verification of splitting is dealt with in [Sut06], and the application of definitions does not arise in this work. To verify that assumptions have been discharged, the assumptions must be added as conditions on the non-assumption parents, forming an implication from the conjunction of the assumptions to the parents. The inferred formula must then be proved (the SZS status must be `thm`) from this implication. In GDV this overall verification obligation is broken down into an equivalent set of small verification obligations, to prove the inferred formula from each of the negated assumptions, and from the non-assumption parents. The benefit of breaking down this verification into the individual components is that a failure to discharge any of these verification obligations provides precise information regarding the fault in the derivation.

General semantic verification has not been modified from that described in [Sut06]. The most common type of verification obligation that must be discharged is, as explained above, to prove that the inferred formula is a theorem of its parents (SZS status `thm`). While this can always be directly attempted by a trusted ATP system, depending on the nature of the inference step the obligation problem might be quite difficult, and the discharge attempt might fail due to resource limits. In these cases GDV tries to disprove the obligation using model finding ATP systems, to establish that the inference is really faulty. Two other types of obligations that arise are to prove that the inferred formula is a countertheorem of its parents (SZS status `cth`), e.g., in the case of a negated conjecture, and to prove a satisfiability-bijection between the parents and inferred formula, e.g., in Skolemization steps. At this stage GDV does only incomplete verification of satisfiability-bijection inferences. Such steps are therefore avoided

in the translation of Mizar proofs, and the additional Henkin axioms are used instead for Mizar steps (**Let**, **Consider**) analogous to ATP Skolemization.

4 Verification of the MPTP Challenge Problems

The MPTP system was used to generate the derivations corresponding to the 252 problems in the *bushy* division of the MPTP challenge (the *chainy* division problems simply add redundant lemmas to the bushy versions). The derivations, together with the corresponding MPTP challenge problems, were given to GDV for verification. GDV was configured to use E 0.99 [Sch02] for finding proofs, and Paradox 2.0b [CS03] for disproofs. A 10s time limit was imposed on each E and Paradox run. All the structural checks (notably the propagation of assumptions) were successful, verification of leaves (checking that they either correspond to the problem, or are introduced definitions, tautologies, or assumptions) was successful, and all the (1303) ATP problems originating from checking the correctness of discharging assumptions were successfully proved.

From the 6765 ATP tasks corresponding to the Mizar atomic inference steps, 60 could not be solved in the 10s time limit by E, but none were found to be countersatisfiable by Paradox. Such timeouts can often be resolved by heuristic pruning of the (potentially superfluous) background formulae added as axioms by the MPTP translation. Therefore the MaLARea system, developed for doing such heuristic pruning along with time limit manipulations in a systematic and automated way [Urb07], was used. The 60 unsolved ATP tasks came from 27 derivations, i.e., at this point 225 of the problems had been verified. To create a wide training set for MaLARea's machine learning based heuristic pruning of axioms, all 759 of the ATP tasks generated by GDV during verification of the 27 derivations were put into the MaLARea's initial set of problems. MaLARea was then run with the maximal time limit set to 64 seconds, and the maximal axiom limit set to 64 axioms (these values were estimated empirically). The result of the MaLARea run was the solution of 742 tasks (this took about 4 hours), solving 43 of the 60 previously unsolved tasks, and leaving 17 tasks unsolved. This was a sufficiently low number to inspect the remaining problems manually.

One task contained quite large formulae, and could be solved easily by running E with more aggressive introduction of definitions during clausification. Thirteen tasks were solved by inspecting the original Mizar proofs and pruning unnecessary axioms by hand. It turns out that the 3 remaining ATP tasks (all coming from checking the derivation of Theorem 29 in Mizar article **YELLOW_0**) might be countersatisfiable. This was traced back to a very rare case when the MPTP algorithm for addition of background formulae was incomplete. Once one additional background formula was added manually to these three ATP tasks, all of them became easily provable. Also when this additional background formula was added to the original MPTP challenge problem, it became solvable directly by some ATP systems. This means that the original challenge problem is very likely countersatisfiable – this would have to be confirmed by finding a model. The original Mizar proof cannot be used to guide a proof due to the missing

background formula, and the problem does not seem to have alternative solution without it. The remaining 251 MPTP Challenge problems were thus shown to be solvable, i.e., it has been verified that the problem conjectures really are theorems of the problem axioms. Detailed documentation of all the steps of the verification process described here is available online.¹⁰

5 Conclusions and Future Work

We have defined and implemented a translation of the Mizar natural deduction formalism to a simpler sequent-like formalism, encoded in the TPTP language (which required a mild extension to deal with assumptions). Proofs of all MML theorems have been exported into this format. The GDV verification tool has been extended to fully verify these derivations. Together with GDV's checking of the introduced derivation leaves, and a simple "correct by construction" argument about the introduced Henkin axioms, this provides a framework for full independent verification of the translated MML proofs. The framework was tested on 252 problems selected for the MPTP challenge, verifying the structural correctness of all the translated derivations, and using the E 0.99 system to automatically verify over 99% of the proof obligations coming from the atomic inference steps. 72% (43) of the remaining obligations were solved by the MaLAREa system, 23% (14) were solved by expert advice, and 5% (3) were found to be very likely to be countersatisfiable, uncovering a rare case where the MPTP translation did not add enough Mizar background axioms for solution of the corresponding ATP problems. The result is a verification of 251 of the MPTP challenge problems, and a verification of the remaining problem corrected by addition of the missing background axiom.

An obvious future step is to verify the whole MML from its axioms. This will require some extensions of the MPTP system, so that the background theories are correctly introduced (most of them have to be proved in Mizar before they can be used), and not treated as just additional axioms. It would be generally hard for other proof assistants (such as HOL Light and Isabelle) to import and verify the translated derivations, because their automated theorem proving is quite weak. However, it is likely that these systems will be able to check the detailed proof objects created by ATP systems such as E and Otter. So other future work is to instruct ATP systems to create detailed proof objects during the GDV verification, and use them afterwards for refining the translated derivations to the point where other proof assistants will be able to import them. A similar import of the ATP proofs back to Mizar would also be useful (a basic translator for detailed Otter proof objects has been around since 2003). The imported proofs could be cross-verified by the Mizar proof checker to strengthen the confidence (which is necessarily empirical) in the translation and verification processes. Finally, it is planned to make this this verification process available as a push-button tool for Mizar authors.

¹⁰ <http://lipa.ms.mff.cuni.cz/~urban/MPTPChallengeVerification.tar.gz>

References

- [BDH⁺99] Bertot, Y., Dowek, G., Hirschowitz, A., Paulin, C., Théry, L. (eds.): TPHOLs 1999. LNCS, vol. 1690. Springer, Heidelberg (1999)
- [CS03] Claessen, K., Sorensson, N.: New Techniques that Improve MACE-style Finite Model Finding. In: Baumgartner, P., Fermueller, C. (eds.) Proceedings of the CADE-19 Workshop: Model Computation - Principles, Algorithms, Applications (2003)
- [Dav81] Davis, M.: Obvious logical inferences. In: Hayes, P.J. (ed.) IJCAI, pp. 530–531. William Kaufmann (1981)
- [DFS06] Denney, E., Fischer, B., Schumann, J.: An empirical evaluation of automated theorem provers in software certification. *International Journal on Artificial Intelligence Tools* 15(1), 81–108 (2006)
- [FS06] Furbach, U., Shankar, N. (eds.): IJCAR 2006. LNCS (LNAI), vol. 4130, pp. 17–20. Springer, Heidelberg (2006)
- [Hal04] Hales, T.C.: Formalizing the proof of the kepler conjecture. In: Slind, K., Bunker, A., Gopalakrishnan, G.C. (eds.) TPHOLs 2004. LNCS, vol. 3223, p. 117. Springer, Heidelberg (2004)
- [Har96] Harrison, J.: A Mizar Mode for HOL. In: von Wright, J., Harrison, J., Grundy, J. (eds.) TPHOLs 1996. LNCS, vol. 1125, pp. 203–220. Springer, Heidelberg (1996)
- [Jas34] Jaskowski, S.: On the rules of suppositions. *Studia Logica*, 1 (1934)
- [McL06] McLaughlin, S.: An interpretation of Isabelle/HOL in HOL Light. In: Furbach and Shankar [FS06], pp. 192–204
- [MS00] McCune, W., Shumsky, O.: System description: IVY. In: McAllester, D. (ed.) Automated Deduction - CADE-17. LNCS, vol. 1831, pp. 401–405. Springer, Heidelberg (2000)
- [NB02] Naumowicz, A., Bylinski, C.: Basic elements of computer algebra in MIZAR. *Mechanized Mathematics and Its Applications* 2 (2002)
- [NB04] Naumowicz, A., Bylinski, C.: Improving mizar texts with properties and requirements. In: Asperti, A., Bancerek, G., Trybulec, A. (eds.) MKM 2004. LNCS, pp. 3119, pp. 290–301. Springer, Heidelberg (2004)
- [OS06] Obua, S., Skalberg, S.: Importing HOL into Isabelle/HOL. In: Furbach and Shankar [FS06], pp. 298–302
- [Pel99] Pelletier, F.J.: A brief history of natural deduction. *History and Philosophy of Logic* 20, 1–31 (1999)
- [RT99] Rudnicki, P., Trybulec, A.: On equivalents of well-foundedness. *J. Autom. Reasoning* 23(3-4), 197–234 (1999)
- [Rud87] Rudnicki, P.: Obvious inferences. *J. Autom. Reasoning* 3(4), 383–393 (1987)
- [Rud92] Rudnicki, P.: An overview of the Mizar project. In: 1992 Workshop on Types for Proofs and Programs, Chalmers University of Technology, Bastad, pp. 311–332 (1992)
- [RV02] Riazanov, A., Voronkov, A.: The design and implementation of VAMPIRE. *Journal of AI Communications* 15(2-3), 91–110 (2002)
- [Sch02] Schulz, S.: E – a brainiac theorem prover. *Journal of AI Communications* 15(2-3), 111–126 (2002)
- [SS98] Sutcliffe, G., Suttner, C.B.: The TPTP problem library: CNF release v1.2.1. *Journal of Automated Reasoning* 21(2), 177–203 (1998)

- [Sut06] Sutcliffe, G.: Semantic Derivation Verification. *International Journal on Artificial Intelligence Tools* 15(6), 1053–1070 (2006)
- [Sym99] Syme, D.: Three tactic theorem proving. In: Bertot et al. [BDH+99], pp. 203–220
- [SZS04] Sutcliffe, G., Zimmer, J., Schulz, S.: TSTP Data-Exchange Formats for Automated Theorem Proving Tools. In: Sorge, V., Zhang, W. (eds.) *Distributed and Multi-Agent Reasoning. Frontiers in Artificial Intelligence and Applications*, IOS Press, Amsterdam (2004)
- [Urb04] Urban, J.: MPTP - motivation, implementation, first experiments. *Journal of Automated Reasoning* 33(3-4), 319–339 (2004)
- [Urb06a] Urban, J.: MPTP 0.2: Design, implementation, and initial experiments. *J. Autom. Reasoning* 37(1-2), 21–43 (2006)
- [Urb06b] Urban, J.: XML-izing Mizar: making semantic processing and presentation of MML easy. In: Kohlhase, M. (ed.) *MKM 2005. LNCS (LNAI)*, vol. 3863, pp. 346–360. Springer, Heidelberg (2006)
- [Urb07] Urban, J.: MaLAREa: a metasystem for automated reasoning in large theories. In: Sutcliffe, G., Urban, J., Schulz, S. (eds.) *ESARLT: Empirically Successful Automated Reasoning in Large Theories. CEUR Workshop Proceedings. CEUR*, vol. 257, pp. 45–58 (2007)
- [UTSP07] Urban, J., Trac, S., Sutcliffe, G., Puzis, Y.: Combining Mizar and TPTP semantic presentation tools. In: *Proceedings of the Mathematical User-Interfaces Workshop 2007* (2007), <http://www.activemath.org/workshops/MathUI/07/proceedings/Urban-et-al-MizarIDV.html>
- [WBH⁺02] Weidenbach, C., Brahm, U., Hillenbrand, T., Keen, E., Theobald, C., Topic, D.: SPASS version 2.0. In: *CADE*, pp. 275–279 (2002)
- [Wen99] Wenzel, M.: Isar - a generic interpretative approach to readable formal proof documents. In: Bertot et al. [BDH+99], pp. 167–184
- [Wie00] Wiedijk, F.: CHECKER - notes on the basic inference step in Mizar (2000), available at <http://www.cs.kun.nl/~freek/mizar/by.dvi>
- [Zam99] Zammit, V.: On the implementation of an extensible declarative proof language. In: Bertot et al. [BDH+99], pp. 185–202